6. Datenbankinhalte ändern

SQL verwaltet nicht nur die Abfragen einer Datenbank. Die gesamte Datenbank wird durch SQL erstellt und verändert. Dazu gibt es weitere Befehle, die analog zum SELECT-Befehl genutzt werden können: Mit **INSERT** werden neue Daten in die Datenbank eingefügt, mit **UPDATE** können vorhandene Daten verändert werden und **DELETE** löscht Daten.

6.1 Einfügen

Das Einfügen von Daten geschieht über **INSERT**. Dabei müssen die entsprechende Tabelle und deren Attribute nach **INTO** angegeben werden. Die Liste der Attribute muss nicht vollständig sein, ausgelassene Attribute werden mit NULL oder ihrem Standardwert (*DEFAULT*) gefüllt. **VALUES** leitet dann die Werte für das neue Objekt ein. Es können mehrere Objekte gleichzeitig eingefügt werden.

AUFBAU

```
INSERT INTO <Klasse>
  (<Liste der Attribute>)
VALUES
  (<Werte vom 1. Objekt>),
  (<Werte vom 2. Objekt>);
```

BEISPIEL · ERSTELLE DEN NEUEN USER GÜNTHER

```
INSERT INTO users
  (username, name, email, password,
   gender, birthday, city, country,
   centimeter)
VALUES
  ('guenther37', 'Günther Müller',
   'guenther@mail.test', '123passwort',
   'male', '1987-07-07', 'Kiel',
   'Deutschland', 178);
```

Statt fester Werte kann INSERT auch das Ergebnis einer Abfrage einfügen – wieder eine Kapselung. Dann fällt VALUES weg:

ALLE USER AUS KIEL FOLGEN DEM USER MIT DER ID 71

```
INSERT INTO follows (follower_id, following_id)
SELECT id, 71
FROM users
WHERE city = 'Kiel'
```

Aufgabe 9

Notiere für die folgenden Aufgaben einen SQL-Befehl und führe ihn aus. Kontrolliere das Ergebnis in der App.

a) Füge eine fiktive Person zu deiner Datenbank hinzu.

b) Überprüfe, welche id diese Person bekommen hat, obwohl du sie nicht angegeben hast. Welche Werte stehen in role und created_at? Woher kommen sie? (Schau ins Diagramm.)

c) Melde dich in der App mit dieser Person an. Es klappt nicht! Aber warum? Finde heraus, woran es liegt.

d) Schreibe unter ein Foto deiner Wahl Kommentare im Namen von mindestens drei verschiedenen Usern. Schau in der App nach: Kann man erkennen, dass die Kommentare nicht echt sind?

e) Gönn dir ein paar mehr Follower: Alle User aus München sollen deinem Admin-Konto folgen. Erstelle dafür keine neuen Personen.

f) * Lass 20 User ein Foto deiner Wahl mit 😂 (laugh) liken. Was passiert, wenn einer von ihnen das Foto schon gelikt hat? Lies die Fehlermeldung und erkläre sie mit dem Diagramm.

* nur mit tieferen SQL-Befehlen lösbar (Abschnitt 4)

6.2 Verändern

Zum Verändern von Daten wird der **UPDATE**-Befehl genutzt. Hierbei können einzelne Attribute beliebig neu besetzt werden. Bei welchen Objekten die Daten geändert werden, wird im WHERE-Befehl definiert. **Ohne WHERE werden alle Zeilen geändert!**

AUFBAU

```
UPDATE <Klasse>
SET <Attribut> = <neuer Wert>,
    <Attribut> = <neuer Wert>, …
WHERE <Bedingung>
```

BEISPIEL · ALLE HAMBURGER ZIEHEN NACH KIEL, WEIL BESSER

```
UPDATE users
SET city = 'Kiel'
WHERE city = 'Hamburg'
```

Aufgabe 10

Notiere für die folgenden Aufgaben einen SQL-Befehl und führe ihn aus.

a) Versuche, das Geschlecht aller User aus Dresden auf „keine Angabe“ zu setzen. Lies die Fehlermeldung: Welche Werte sind erlaubt? Wo im Diagramm steht das? Warum ist so eine Einschränkung sinnvoll?

b) Suche dir 10 Bilder von anderen Usern deiner Datenbank aus und mache sie zu deinen Bildern. Wie sieht dein Profil danach aus?

c) * ★ Lenke alle Likes auf eines deiner Bilder um. Warum klappt das nicht? *Tipp: Schau dir den Primärschlüssel von likes an.* Schaffst du es, wenigstens die Likes umzulenken, die möglich sind?

d) Ersetze die Körpergröße aller User durch `FLOOR(RAND() * 50) + 150`. Was erzeugt dieser Befehl? Führe ihn zweimal aus und vergleiche.

e) * User, die ihre Boote fotografiert haben, möchten ihren Wohnort durch „auf hoher See“ ersetzen. Zwei Admins schlagen verschiedene Befehle vor (rechts). Welcher erfüllt den Wunsch besser? Prüfe es erst mit SELECT.

BEFEHL 1

```
UPDATE users
SET city = 'auf hoher See'
WHERE id IN
  (SELECT user_id FROM photos
   WHERE description
     LIKE '%boot%')
```

BEFEHL 2

```
UPDATE users
SET city = 'auf hoher See'
WHERE id IN
  (SELECT photos.user_id
   FROM photos
   JOIN photo_tag ON …
   JOIN tags ON …
   WHERE tags.name = 'boot')
```

f) hugo.ranked hat sein Passwort vergessen. Setze es neu und melde dich danach als Hugo an.

g) Der Sicherheitsbeauftragte vermutet, dass manche User sehr schwache Passwörter haben. Finde mit `PASSWORD_VERIFY` heraus, wer „123456“ benutzt. Probiere auch „passwort“ und „hallo123“. Erkläre: Warum kann man den Hash nicht zurückrechnen, die Passwörter aber trotzdem finden?

* nur mit tieferen SQL-Befehlen lösbar (Abschnitt 4) · ★ zum Knobeln

6.3 Löschen

Löschen funktioniert auf die gleiche Art und Weise, nur ohne den SET-Befehl. Auch hier gilt: **Ohne WHERE wird alles gelöscht!**

AUFBAU

```
DELETE FROM <Klasse>
WHERE <Bedingung>
```

BEISPIEL · ALLE FOTOS DES USERS MIT DER ID 3 WERDEN GELÖSCHT

```
DELETE FROM photos
WHERE user_id = 3
```

NACHGEDACHT · **Was passiert mit den Kindern?**

Löschst du ein Foto, gehören die Likes und Kommentare daran zu einem Foto, das es nicht mehr gibt. In YourSozialNet sind die Fremdschlüssel deshalb mit **ON DELETE CASCADE** angelegt: Die abhängigen Zeilen werden automatisch mitgelöscht.

Aufgabe 11

Notiere für die folgenden Aufgaben einen SQL-Befehl. Prüfe vorher mit SELECT, was du löschst!

a) Lösche alle Lisas aus deiner Datenbank. Vergleiche vorher `name LIKE 'Lisa%'` und `name LIKE '%Lisa%'`. Wie viele Lisas gibt es wirklich?

b) Lösche alle Bilder von Alexander Achen. Was passiert mit den Likes und Kommentaren an seinen Bildern?

c) ★ Lösche alles, was mit Henriette Zimmerman in Verbindung steht, ihren Account jedoch nicht: alle Fotos, alle Kommentare, alle Follower, alle Abos und ihr Profilbild. *Tipp: Das Profilbild ist kein eigenes Objekt.*

d) Lösche alle Personen unter 18. Wie viele Personen waren das?

e) * Verschwörungstheorien über Echsenmenschen werden in deinem Netzwerk nicht geduldet. Lösche alle Kommentare der Echsen-Theoretiker – aber nur diese! Sofias harmlose Bartagame und die Antworten anderer User sollen bleiben. Vergleiche dazu die beiden Befehle rechts mit SELECT (vgl. Aufgabe 5 h).

BEFEHL 1

```
DELETE FROM comments
WHERE body LIKE '%Echse%'
```

BEFEHL 2

```
DELETE FROM comments
WHERE body LIKE '%Echse%'
  AND user_id IN
  (SELECT id FROM users
   WHERE username IN
   ('mondrueckseite',
    'echsenwache'))
```

f) Lösche den User aaron183. Zähle vorher und nachher alle Kommentare. Warum fehlen danach Kommentare, obwohl du in comments gar nichts gelöscht hast?

* nur mit tieferen SQL-Befehlen lösbar (Abschnitt 4) · ★ zum Knobeln